SQL migration schema: separate billing accounts from marketplace partners and track GMV with a transaction classification table

Computer
Transaction classification data model: schema, indexing strategy, and partition rules for high-volume marketplace GMV tracking
Computer
Walk through the SQL migration line by line
How should we resolve the canonical organization link
Draft the transaction classification table for GMV

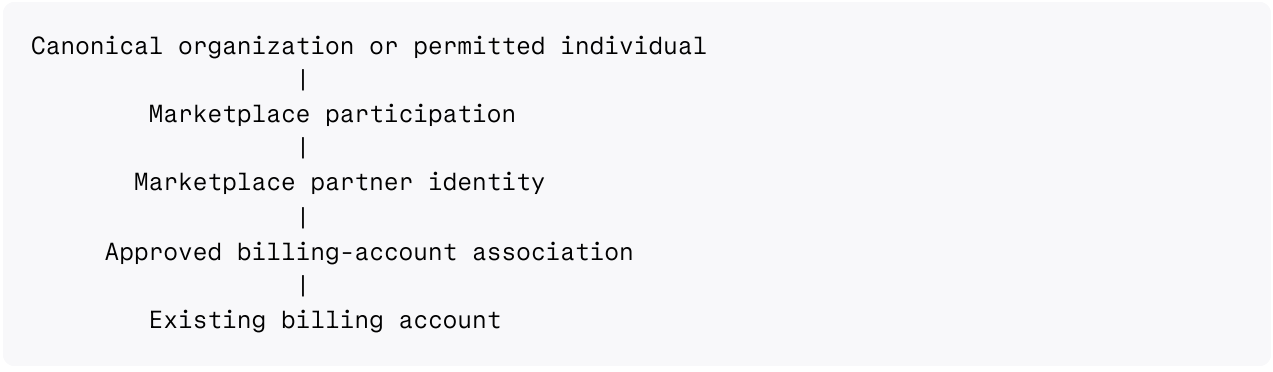
Keep billing accounts, marketplace participation, and canonical organization identity as three separate concepts. For GMV, use a source-attributed transaction record plus a versioned classification history; calculate reporting from the latest approved classification as of an explicit cutoff.

For high volume, preserve a small, unpartitioned identity and deduplication layer, then partition large event or reporting tables only when measured workloads justify it. Do not simply add `PARTITION BY` to the current draft: PostgreSQL requires partitioned-table primary and unique constraints to include the partition key, which changes duplicate protection and foreign-key design.[1]

Below is a walkthrough of the migration, the organization-link decision, a concrete classification table, and indexing/partition rules. All SQL remains **review-only**: nothing has been applied or PostgreSQL-tested.

1. Resolve the canonical organization link

Separate the three identities



Concept	Meaning	Must not imply
Canonical organization	The business or other organization being represented	Marketplace participation or billing responsibility
Marketplace partner	A scoped seller/provider relationship	Ownership of every linked billing account
Billing account	Responsibility for charges and billing administration	Seller identity, GMV attribution, or access authority

An integration provider is also not automatically a marketplace seller. The same organization might hold both roles, but those relationships need separate evidence.

Recommended resolution process

1. Inventory existing commercial identity records.
   Inspect customer, account, employer, vendor, partner, contract, and organization relationships—not just table names.
2. Identify the authoritative entity master.
   Determine which existing record actually represents a commercial organization, rather than an institution account, student club, integration adapter, or billing arrangement.
3. Preserve distinct relationships.
   Add marketplace participation to that entity; do not copy its legal identity into a marketplace-specific master.
4. Link verified external identifiers.
   Use source namespace plus external identifier. Do not merge organizations by display name alone.
5. Review conflicts.
   Rebrands, subsidiaries, shared billing, acquisition, and accidental duplicates require explicit merge/split decisions.
6. Approve scope and lifecycle rules.
   A partner can operate across institutions while each institutional participation remains separately authorized.

Your project reconciliation explicitly warns that reference migrations can duplicate existing schemas and permission predicates. It requires extending the repository's canonical capability system instead.[2]

What to change in the current draft

The drafted `marketplace_partners` table currently combines source identity with school-scoped participation. That is acceptable as a bounded staging scaffold, but before production:

- Rename its meaning explicitly to "marketplace participation," or separate participation from global partner identity.
- Replace an unverified organization reference only after the authoritative table and key type are confirmed.
- Avoid treating each campus participation as a different legal organization.

- Decide whether individual sellers are supported. If they are, use a separately reviewed person relationship with appropriate privacy boundaries.
- Do not add an unenforced `entity_type + entity_id` pair as a substitute for real referential integrity.

A proposed structure—not asserted existing tables—is:

```
Existing canonical business entity
           |
Marketplace partner identity
           |
Marketplace participation by scope
           |
Transactions and approved billing links
```

2. Walk through the migration in execution order

`BEGIN`

Starts the schema change as a transaction. The ordinary table and index statements in the draft can roll back together on failure.

Keep online index-building changes separate when they require `CREATE INDEX CONCURRENTLY`; those cannot simply be inserted into this transactional block.

A. `marketplace_partners`

Line or clause	Purpose	Remaining concern
`id uuid PRIMARY KEY`	Stable internal identifier	UUID alone does not establish canonical organization identity
`school_id text REFERENCES schools(id)`	Uses the previously verified school key type	Scope must still be authorized
`source_namespace`	Identifies the source system or registry	Define its vocabulary and collision rules
`source_partner_key`	Carries the source identifier	Avoid unnecessary personal identifiers
`display_name`	Human-readable label	Not suitable for deduplication
`recorded_at`	Records ingestion time	Not the same as relationship effective time
Unique source identity	Prevents duplicate scoped partner imports	Conflicting replays still need investigation
`UNIQUE(school_id, id)`	Supports composite foreign keys	Useful for cross-scope integrity

B. `marketplace_partner_billing_links`

This is an association, not a billing account or money-movement record.

Clause	Purpose
`partner_id`	Identifies the marketplace participation
`billing_account_id`	References the existing billing account
`purpose`	Distinguishes fee billing, payout reference, and sponsorship
`valid_from`, `valid_until`	Establishes effective dates
`authorization_reference`	Identifies approval evidence
Composite partner foreign key	Prevents linking a partner under the wrong school
Source-link unique key	Prevents duplicate imported associations

The original draft does not yet enforce association overlap, revocation, or account-scope authorization. Those are release blockers, not optional polish.

Also, a text approval reference is not proof of approval. The trusted command must verify the referenced decision.

C. `marketplace_transactions`

This table holds the stable transaction grain.

Field	Meaning
Source namespace/key	Durable identity and duplicate protection
`occurred_at`	Source transaction occurrence
`currency`	Currency identifier
`merchandise_minor`	Merchandise amount before seller discount
`discount_minor`	Seller discount
`tax_minor`, `shipping_minor`	Separately retained components
Evidence reference	Source provenance
`recorded_at`	When Semester received the record

Amounts are in currency minor units, not universally cents.

The discount constraint prevents:

```
discount > merchandise amount
```

It does not establish that all financial components reconcile to a payment. That requires adapter and reconciliation tests.

D. `marketplace_transaction_classifications`

This determines whether a transaction belongs in confirmed GMV.

The key separation is:

```
What happened financially?
        ≠
How should it be reported?
```

Classification revisions can change reporting without rewriting the original transaction.

E. `marketplace_transaction_adjustments`

The draft stores separate events for:

- Merchandise refunds.
- Commissions.
- Commission reversals.
- Variable costs.

Source event uniqueness prevents duplicate events. It does not verify event authenticity, currency, maximum refundable value, or the accuracy of cost allocation.

The query aggregates adjustments before joining them to transactions, preventing fee/refund multiplication through join fanout.

F. Index statements

These support transaction-period queries, latest classification lookup, and adjustment retrieval. Their usefulness must be tested against the actual query plan.

G. RLS, revokes, and grants

The draft enables RLS, removes browser access, and grants only `SELECT` and `INSERT` to the trusted service role.

Important limitations:

- Service-role operations bypass RLS.
- Trusted ingestion must enforce authorization and scope.
- Grants alone do not establish immutable history against database owners.
- No browser reporting policy or authorized reporting RPC has been implemented.

`COMMIT`

Completes the schema transaction.

The parameterized reporting query after `COMMIT` is a separate reference query. Do not paste the entire file into a migration runner without separating that query from the migration statements.

3. Transaction classification table for GMV

The following refines the earlier draft by adding explicit supersession, policy-effective time, source evidence, and command identity.

It assumes the proposed unpartitioned `marketplace_transactions` table already exists with `UNIQUE(school_id, id)`.

```
CREATE TABLE public.marketplace_transaction_classifications (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),

  school_id text NOT NULL,
  transaction_id uuid NOT NULL,

  revision integer NOT NULL
    CHECK (revision > 0),

  supersedes_revision integer,

  classification text NOT NULL CHECK (
    classification IN (
      'marketplace_merchandise',
      'platform_subscription',
      'donation',
      'deposit',
      'tax_only',
      'unclassified'
    )
  ),

  fulfillment_state text NOT NULL CHECK (
    fulfillment_state IN (
      'pending',
      'fulfilled',
      'cancelled',
      'failed',
      'not_applicable'
    )
  ),

  fulfilled_at timestamptz,

  decision text NOT NULL CHECK (
    decision IN ('included', 'excluded', 'review')
  ),
```

```
  reason_code text NOT NULL
    CHECK (length(reason_code) > 0),

  policy_version text NOT NULL
    CHECK (length(policy_version) > 0),

  effective_at timestamptz NOT NULL,

  evidence_reference text NOT NULL
    CHECK (length(evidence_reference) > 0),

  decision_authority_reference text NOT NULL
    CHECK (length(decision_authority_reference) > 0),

  command_key text NOT NULL
    CHECK (length(command_key) > 0),

  recorded_at timestamptz NOT NULL DEFAULT now(),

  FOREIGN KEY (school_id, transaction_id)
    REFERENCES public.marketplace_transactions(school_id, id),

  UNIQUE (school_id, transaction_id, revision),

  UNIQUE (school_id, command_key),

  FOREIGN KEY (
    school_id,
    transaction_id,
    supersedes_revision
  )
    REFERENCES public.marketplace_transaction_classifications(
      school_id,
      transaction_id,
      revision
    ),

  CHECK (
    (revision = 1 AND supersedes_revision IS NULL)
    OR
    (
      revision > 1
      AND supersedes_revision IS NOT NULL
      AND supersedes_revision = revision - 1
    )
  ),

  CHECK (
    (fulfillment_state = 'fulfilled')
    =
    (fulfilled_at IS NOT NULL)
  ),

  CHECK (
    decision <> 'included'
    OR (
```

```
      classification = 'marketplace_merchandise'
      AND fulfillment_state = 'fulfilled'
      AND fulfilled_at IS NOT NULL
    )
  )
);
```

What the database enforces

- The transaction exists under the same school.
- Revision numbers are unique per transaction.
- Later revisions reference their immediate predecessor.
- Included records are fulfilled marketplace merchandise.
- Duplicate command keys fail.

What the command still must enforce

- Authorized actor and approved source.
- Current policy version.
- Valid evidence and approval references.
- Serialized revision creation under concurrent requests.
- Duplicate command payload comparison.
- Business-valid effective dates.
- Append-only behavior and audit.
- Correction notification and metric refresh.

The unique constraints reject a race; they do not themselves provide a complete idempotent retry response.

Historical reporting semantics

Use separate timestamps:

Timestamp	Answers
`occurred_at`	When did the source transaction occur?
`fulfilled_at`	Which fulfillment cohort contains it?
`effective_at`	When does this classification apply?
`recorded_at`	When did Semester know about this revision?

A historical report should select the latest applicable revision using both an effective-time boundary and a knowledge cutoff.

Do not filter to `decision = 'included'` before selecting the latest revision. Otherwise, an older included revision could survive after a later exclusion.

4. Indexing strategy

Latest revision lookup

The unique constraint already creates an index on:

```
(school_id, transaction_id, revision)
```

That can support a reverse revision scan for one transaction. Do not automatically add an equivalent descending index without testing.

PostgreSQL automatically creates indexes for primary-key and unique constraints.[3]

Scope and transaction-period queries

```
CREATE INDEX marketplace_transactions_scope_time_idx
ON public.marketplace_transactions (
  school_id,
  occurred_at,
  partner_id
);
```

This supports scope equality followed by an occurrence-time range. A multicolumn B-tree is generally most efficient when queries constrain its leading columns.[4]

However, your GMV query currently filters by classification `fulfilled_at`, not transaction `occurred_at`. That index alone does not optimize the actual reporting predicate.

High-volume reporting projection

Maintain a derived current-classification/reporting projection with one row per transaction, then index:

```
school_id, fulfilled_at, partner_id
```

Its queryable fields should include classification revision, eligibility, currency, gross/net components, and completeness metadata.

This is a rebuildable read model—not another authoritative transaction ledger.

Review queue

A partial index can support review work:

```
CREATE INDEX marketplace_classification_review_idx
ON public.marketplace_transaction_classifications (
  school_id,
  recorded_at,
  transaction_id
)
WHERE decision = 'review';
```

This includes historical review revisions too. A current-review queue must suppress transactions whose later revision resolved the review.

Partial indexes contain only rows satisfying their predicate; the query must use a compatible condition.[5]

Adjustment retrieval

```
CREATE INDEX marketplace_adjustments_transaction_time_idx
ON public.marketplace_transaction_adjustments (
  school_id,
  transaction_id,
  effective_at
);
```

Avoid adding a broad JSONB GIN index merely because source metadata exists. Index actual query predicates, then measure read benefit against write and storage costs.

5. Partition rules for high volume

Recommended starting point

Keep these unpartitioned initially:

- Canonical partner/participation identities.
- Billing associations.
- Transaction identity and source deduplication.
- Latest/current classification pointer.
- Global source-event deduplication.

Partition these when demonstrated scale warrants it:

- Classification/event history.
- Large adjustment event history.
- Fulfillment-based reporting facts.

PostgreSQL notes that partitioning benefits depend on table size and access patterns; poor partition design can increase planning and maintenance overhead. [1]

The uniqueness trap

If classification history is partitioned by `recorded_at`, PostgreSQL cannot directly enforce:

```
UNIQUE (school_id, transaction_id, revision)
```

across partitions unless the constraint includes `recorded_at`. Adding that timestamp changes uniqueness semantics: the same revision could appear in different partitions.

Therefore, retain an unpartitioned revision registry or equivalent guarded identity layer to enforce:

```
transaction + revision → exactly one history record
```

The partitioned history then references that durable identity. This must be inserted atomically through the reviewed command. [1]

Choosing partition keys

Table	Candidate key	Reason
Reporting facts	Fulfillment month	Matches GMV reporting cohorts
Classification history	Recorded month	Supports ingestion/history maintenance
Adjustment events	Recorded or effective month	Choose based on actual workload and correction semantics
Identity/deduplication	No time partition initially	Preserves global duplicate detection

Do not partition a current classification table on a date that changes when an assessment is corrected without explicitly designing row movement.

Operational rules

- Use half-open monthly boundaries.
- Define boundaries explicitly in UTC where timestamps are used.
- Pre-create partitions ahead of ingestion.
- Alert on missing partitions.
- Quarantine unsupported dates rather than silently dropping events.
- Preserve old transaction references for late refunds.
- Apply retention and legal-hold review before detach/drop.
- Deny direct application access to child partitions.
- Test parent and direct-child access paths.

- Validate partition pruning with real query plans.

PostgreSQL routes rows to matching partitions and rejects inserts when no appropriate partition exists. Pruning depends on compatible partition-key predicates, not merely adding indexes. [1]

6. Required revisions before approval

The next migration revision should:

1. Resolve commercial organization versus marketplace participation.
2. Separate migration DDL from parameterized reporting SQL.
3. Add reviewed billing-link revocation and overlap semantics.
4. Bind classification commands to existing authorization and audit.
5. Establish concurrent revision/idempotency handling.
6. Add current-state reporting projections without replacing history.
7. Choose partitioning only after representative load tests.
8. Preserve global duplicate protection if history is partitioned.
9. Test late refunds, backdated classifications, and historical cutoffs.
10. Verify that missing classifications are counted as incomplete coverage—not silently treated as zero GMV.

The schema direction is sound, but the organization link, command authorization, and historical reporting semantics need resolution before staging deployment approval. A production-ready design must prove those relationships rather than hide them behind nullable IDs or reporting assumptions.

⁂

1. https://www.postgresql.org/docs/17/ddl-partitioning.html
2. The Main Semester design system (2) copy/docs/system/ROLE_SYSTEM_RECONCILIATION.md
3. https://www.postgresql.org/docs/current/indexes-unique.html
4. https://www.postgresql.org/docs/current/indexes-multicolumn.html
5. https://www.postgresql.org/docs/current/indexes-partial.html
6. https://pgadmin-archive.postgresql.org/pgadmin4/v6.7/docs/pgadmin4-6.7.pdf
7. https://www.postgresql.org/docs/current/indexes.html
8. https://pgadmin-archive.postgresql.org/pgadmin4/v7.6/docs/pgadmin4-7.6.pdf
9. https://neon.com/postgresql/indexes/multicolumn-indexes
10. https://pgadmin-archive.postgresql.org/pgadmin4/v4.20/docs/pgadmin4-4.20.pdf
11. https://postgrespro.com/media/docs/postgrespro/10/en/postgres-A4-fop.pdf
12. https://postgres.ai/blog/20211029-how-partial-and-covering-indexes-affect-update-performance-in-postgresql
13. https://ftp.postgresql.org/pub/pgadmin/pgadmin4/v9.18/docs/pgadmin4-9.18.pdf

14. https://www.slideshare.net/slideshow/postgresql-10-new-features-english/76396890
15. https://commitfest.postgresql.org/59/
16. https://stackoverflow.com/questions/44840259/multi-column-index-vs-seperate-indexes-vs-partial-indexes
17. The Main Semester design system (2) copy/handoff/semester-platform/supabase/migrations/090_native_finance.sql.txt
18. The Main Semester design system (2) copy/handoff/semester-platform/supabase/migrations/030_marketplace.sql.txt
19. The Main Semester design system (2) copy/handoff/semester-platform/supabase/migrations/040_ops.sql.txt
20. https://www.postgresql.org/docs/current/sql-createindex.html